

B.Sc. (Data Science)

DS-151-T : Advanced Python Programming

Unit3: Exception Handling

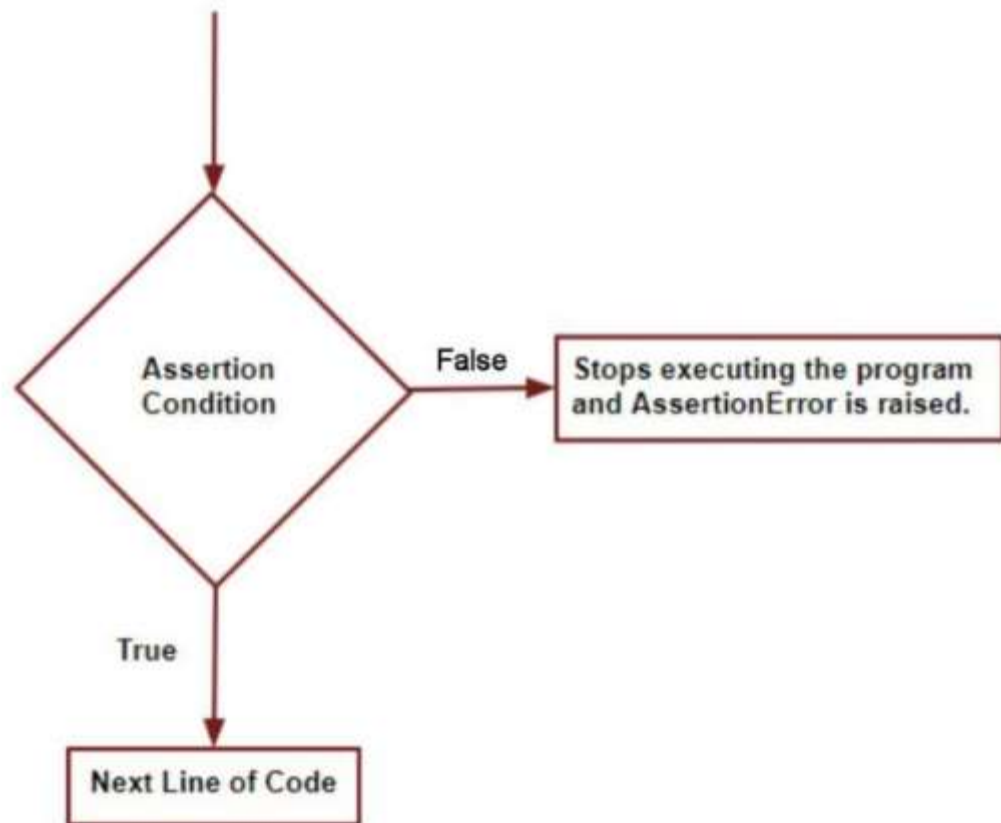
Mrs. Reshma Masurekar

**Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18**

assert

- The `assert` keyword in Python is a tool that evaluates the validity of a statement. In simpler terms, it is a boolean expression.
- If the `assert` keyword returns `true`, the program continues for execution; however, if the `assert` keyword returns `false` (python `assert false`), the program stops.
- **Syntax:**
`assert condition, <error message>`
- Where,
 - `condition` – A boolean expression that should be true.
 - `message (optional)` – An optional message to be displayed if the assertion fails. Here, the condition is the expression or condition that you want to test, and the message is an optional string that provides additional information about the assertion.

- When the assert statement is encountered, it evaluates the given condition.
- If the condition is true, the program continues its execution without any interruptions. However, if the condition is false, assert stops the program and the assert statement raises an **AssertionError** exception.
- It is also a debugging tool as it halts the program as soon as an error occurs and displays it.
- The primary purpose of using assert statements is to detect logical errors and invalid assumptions in your code.
- statement is used to check types, values of argument and the output of the function.



```
assert 0==0, "This shouldn't throw an error."
```

Output:

No error

Explanation:

- The code uses the assert keyword, a debugging tool in Python.
- assert tests if a condition is true. If it is, the program continues to run.
- If the condition is false, the program stops and throws an AssertionError.
- In this case, the condition is `0==0`, which is true, so no error is thrown.
- The string "This shouldn't throw an error." is the error message that would be displayed if the assertion failed.

```
assert 0==0, "This shouldn't throw an error."
```

Output:

No error

Explanation:

- The code uses the assert keyword, a debugging tool in Python.
- assert tests if a condition is true. If it is, the program continues to run.
- If the condition is false, the program stops and throws an AssertionError.
- In this case, the condition is 0==0, which is true, so no error is thrown.
- The string "This shouldn't throw an error." is the error message that would be displayed if the assertion failed.

```
assert 4>3, "This shouldn't throw an error."  
print("End of Program")
```

Output:

End of Program

```
assert 3>4, "This shouldn't throw an error."  
print("End of Program")
```

Output:

Traceback (most recent call last):

File "C:/Users/admin/AppData/Local/Programs/Python/Python311/assertiondemo.py",
line 1, in <module>

assert 3>4, "This shouldn't throw an error."

AssertionError: This shouldn't throw an error.

```
def divide(a, b):
    assert b != 0, "Division by zero!"
    return a / b

# Test cases
print(divide(10, 2))          # Output: 5.0
print(divide(8, 0))          # AssertionError: Division by zero!
```

5.0

Traceback (most recent call last):

File

"C:/Users/admin/AppData/Local/Programs/Python/Python311/assertiondemo.py", line 7, in <module>

print(divide(8, 0)) # AssertionError: Division by zero!

File

"C:/Users/admin/AppData/Local/Programs/Python/Python311/assertiondemo.py", line 2, in divide

assert b != 0, "Division by zero!"

AssertionError: Division by zero!

Different Types of Assertions in Python

1. Value Assertions
2. Type Assertions
3. Collection Assertions
4. Exception Assertions
5. Boolean Assertions

Value Assertions

- used to check whether a certain value meets specific conditions or criteria.
- These assertions are typically used for debugging and testing purposes.
- They help ensure that the values being used in the program are as expected. If an assertion fails, an **AssertionError** is raised.

Examples

```
assert x == 5:
```

Checks if the value of x is equal to 5.

```
assert len(my_list) > 0:
```

Ensures that the length of my_list is greater than 0.

```
assert result in expected_results:
```

Verifies if result is present in expected_results.

```
x=4
```

```
assert x == 5
```

```
print("End")
```

Output:

```
assert x == 5
```

```
AssertionError
```

```
x=5
```

```
assert x == 5
```

```
print("End")
```

Output:

```
End
```

Type Assertions

- Type assertions are used to verify the type of a variable or expression. They ensure that the expected type is matched, otherwise, an **AssertionError** is raised. Type assertions are particularly useful when working with dynamically typed languages like Python, where the type of a variable can change.

Examples

```
assert isinstance(x, int):
```

Validates that x is an instance of the int type.

```
assert isinstance(my_list, list):
```

Checks if my_list is of type list.

Collection Assertions

- Collection assertions are used to verify properties of collections, such as lists, tuples, sets, or dictionaries.
- These assertions allow you to check if a collection contains specific elements or satisfies certain conditions.

Examples

assert item in my_list:

Verifies if item is present in **my_list**.

assert key in my_dict:

Ensures that key exists in **my_dict**.

Exception Assertions

- Exception assertions are used to test whether a specific exception is raised or not. They are commonly used in unit tests to ensure that exceptions are handled correctly in code.

Examples

`assert_raises(ExceptionType, my_function, arg1, arg2):`

Checks if **my_function** raises an exception of type **ExceptionType** when called with **arg1** and **arg2**.

`assert_raises(ValueError, int, 'abc'):`

Validates that **int('abc')** raises a **ValueError**.

Boolean Assertions

- Boolean assertions are used to check the truthiness of a condition or expression. They ensure that a certain condition evaluates to True, otherwise, an AssertionError is raised.

Examples

`assert condition:`

Evaluates the truthiness of the condition.

`assert x > y:`

Checks if x is greater than y.

raise

- raise keyword is used to throw(raise) an exception if a condition occurs.
- An exception can be raised forcefully by using the raise clause in Python.
- Syntax:

raise Exception("user text")

- Example:

raise Exception("ZeroDivisionError")

- The raise keyword is typically **used inside a function or method, and is used to indicate an error condition.**
- We can throw an exception and immediately halt the running of your programme by using the raise keyword.
- Python looks for the closest exception handler, which is often defined using a try-except block, when an exception is triggered.
- If an exception handler is discovered, its code is performed, and the try-except block's starting point is reached again.
- If an exception handler cannot be located, the software crashes and an error message appears.

Python program to raise an exception divide function.

def divide(a, b):

if b == 0:

raise Exception(" Cannot divide by zero. ")

return a / b

try:

 result = divide(10, 0)

except Exception as e:

print(" An error occurred: ", e)

Raise an error and stop the program if x is lower than 0

```
x = -1
```

```
if x < 0:  
    raise Exception("Sorry, no numbers below zero")
```

Checking whether an integer is odd or even

```
x = 5
```

```
if x % 2 != 0:  
    raise Exception("The number shouldn't be an odd integer")
```

changing the string data' assigned to integer and wrote a try-except clause to raise the ValueError. **The raise error keyword raises a value error with the message "String can't be changed into an integer".**

```
str = 'data '  
try:  
    num = int(str)  
except ValueError:  
    raise ValueError("String can't be changed into integer")
```

Raising an exception Without Specifying Exception Class

```
str = 'data'  
try:  
    num = int(str)  
except:  
    raise
```

Python program to raise an exception use check_age function to checks the age

def check_age(age):

if age < 18:

raise Exception(" Age must be 18 or above. ")

else:

print(" Age is valid. ")

try:

 check_age(17)

except Exception as e:

print(" An error occurred: ", e)

Python program for Handling raising exceptions.

try:

code that might raise an exception

value = int("a")

except ValueError:

code that handles the ValueError exception

print(" Invalid value ")

except TypeError:

code that handles the TypeError exception

print(" Invalid type ")

Custom exception

- Exceptions created by programmers are called as custom exception.
- It allows to define own error conditions and handle them in a more specific and meaningful way.
- Example: withdraw money, age error
- Custom exceptions are defined by creating a new class that is derived from the built in Exception class.

Syntax

```
class CustomError(Exception):  
    ----  
    pass  
  
try:  
    ---  
except CustomError:  
    ---
```

Step1: Create exception class and inherit it from base Exception class i.e. Exception.

```
class AgeError(Exception):
```

```
    print("Raised when person age is less than 18")
```

```
    pass
```

Step2: Raise AgeError exception for particular condition inside try block.

```
try:
```

```
    age=int(input("Enter Person age:"))
```

```
    if(age <18):
```

```
        raise AgeError
```

Step 3: Handle the exception using except block.

```
except AgeError:
```

```
    print("Person is not Eligible to vote")
```


#Python program to raise a user defined exception to check if age is less than 18.

```
class AgeError(Exception):  
    print("Raised when person age is less than 18")  
    pass  
  
try:  
    age=int(input("Enter Person age:"))  
    if(age <18):  
        raise AgeError  
except AgeError:  
    print("Person is not Eligible to vote")  
else:  
    print("Person is Eligible to vote")
```

Output1:

Raised when person age is less than 18
Enter Person age:20
Person is Eligible to vote

Output2:

Raised when person age is less than 18
Enter Person age:10
Person is not Eligible to vote

#Python program for FiveDivision Error Exception.

```
class FiveDivisionError(Exception):
```

```
    print("Error when Divide a number by five")
```

```
    pass
```

```
try:
```

```
    n1=int(input("Enter Number1:"))
```

```
    n2=int(input("Enter Number2:"))
```

```
    if n2==5:
```

```
        raise FiveDivisionError ("Cannot Divide by Five")
```

```
    ans=n1/n2
```

```
    print("Answer: ",ans)
```

```
except (FiveDivisionError,ZeroDivisionError) as e:
```

```
    print(e)
```

```
print("End of Program...")
```

Output1:

Error when Divide a number by five

Enter Number1:10

Enter Number2:2

Answer: 5.0

End of Program...

Output2:

Error when Divide a number by five

Enter Number1:10

Enter Number2:5

Cannot Divide by Five

End of Program...

Output3:

Error when Divide a number by five

Enter Number1:10

Enter Number2:0

division by zero

End of Program...

#Python program for FiveDivison Error Exception.

```
class FiveDivisionError(Exception):  
    def __init__(self):  
        print("Error cannot divide by five")  
    pass  
  
try:  
    n1=int(input("Enter Number1:"))  
    n2=int(input("Enter Number2:"))  
    if n2==5:  
        raise FiveDivisionError  
    ans=n1/n2  
    print("Answer: ",ans)  
except (FiveDivisionError,ZeroDivisionError) as e:  
    print(e)  
print("End of Program...")
```

Output:

```
Enter Number1:10  
Enter Number2:5  
Error cannot divide by five  
  
End of Program...
```